

What are we testing?

Can a coding agent build a working forecast from scratch, and how does that depend on the model?

Does adding Rhiza weather skills or ACCORD libraries improve completion, time or cost?

When the next forecast is needed, does retained code and fitted state reduce the work?

Report completion, forecast error, tokens, time and cost separately. Treat the expected advantages as hypotheses.

The three arms we actually tested

Scratch

Standard scientific Python; the agent builds the workflow.

Rhiza skills

The same Python environment + Rhiza weather skills catalog and scripts.

ACCORD

The same Python environment + africas2s (DeepScale) and Rosetta/acmadDL, with API documentation.

Identical supplied data and budgets. Every arm can retain its own code, inputs and fitted models.

Observed use: Rhiza catalog context, no script calls; ACCORD agents called africas2s. Data retrieval was not tested.

The task: a reusable seasonal rainfall forecast

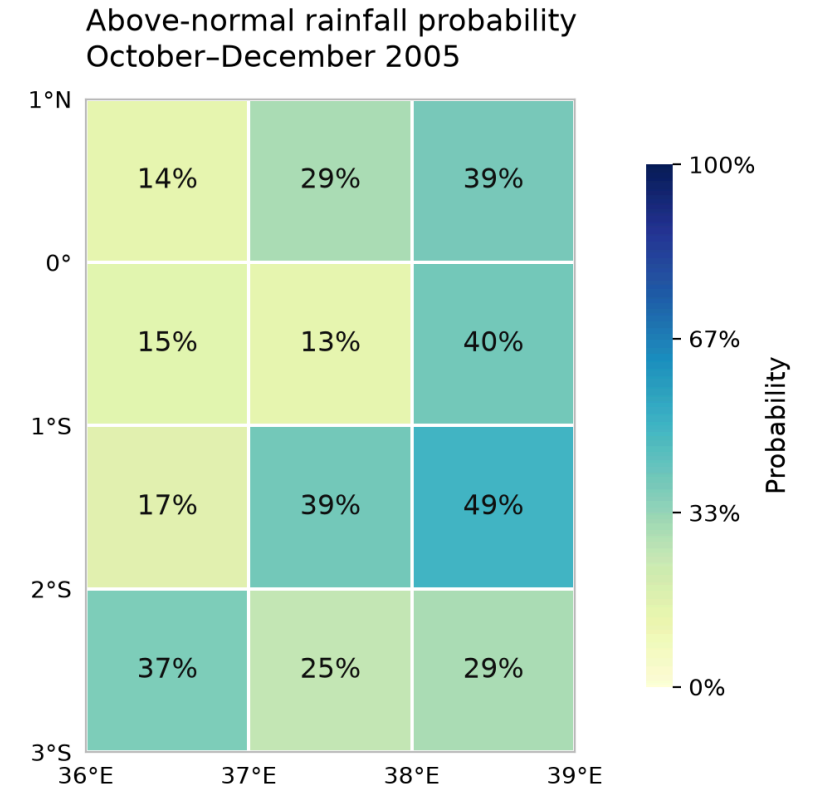
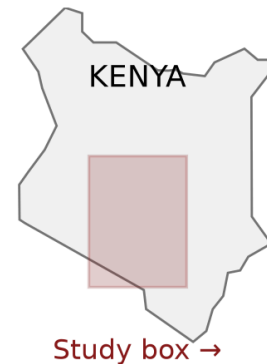
Same data and budgets; scratch, Rhiza skills, or ACCORD libraries (DeepScale / Rosetta).

Turn September-issued ECMWF forecasts and CHIRPS observations into Oct–Dec rainfall probabilities for a 12-cell Kenyan study box.

Train on 1993–2004; predict 2005–06. Save the code and fitted model, then forecast 2007–08 in a fresh conversation.

Check completion and replay, scientific quality with an LLM judge, forecast error, tokens, time and cost.

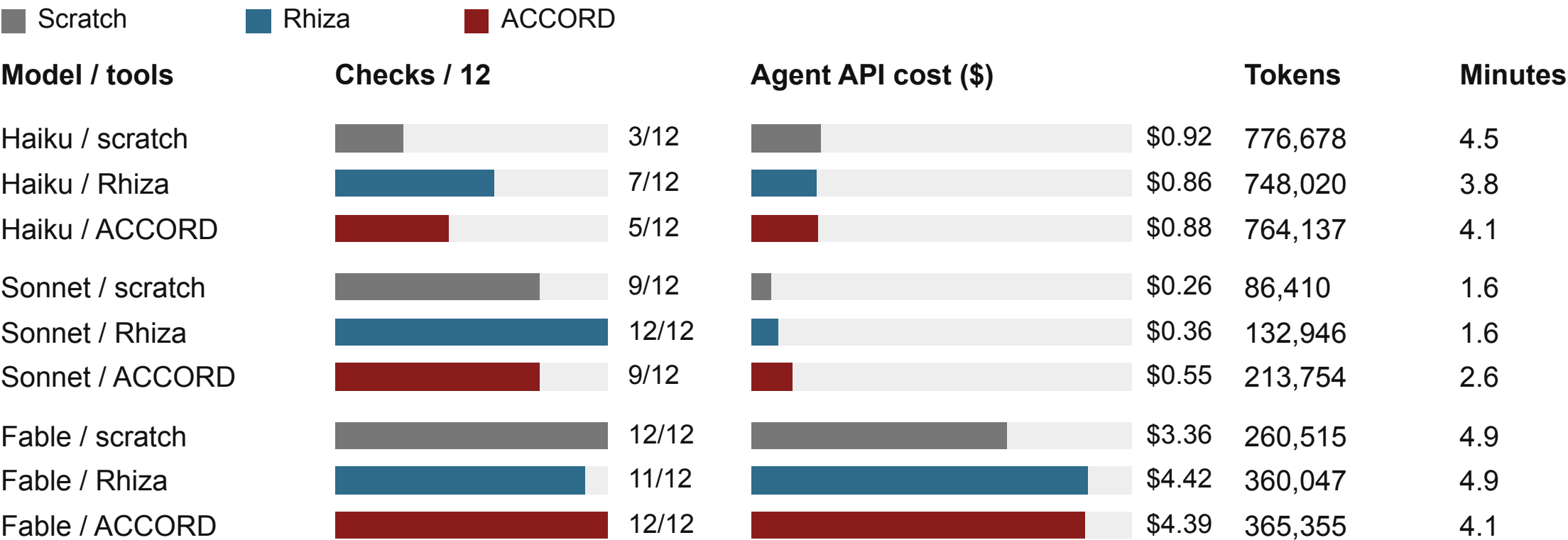
Can stronger agents finish? Do tools save effort? Does saved work help next time?



Example: Fable + ACCORD, historical 2005 forecast; 33% is climatology.

Model capability and tools

Nine initial attempts: three models × three tool environments, with the same task, data and limits.

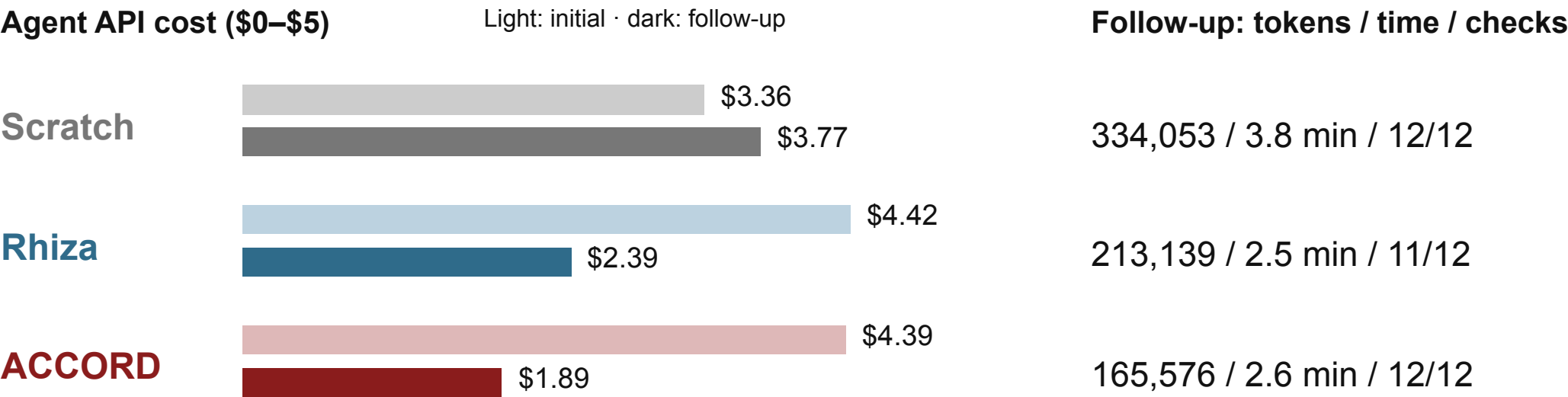


Sonnet + Rhiza matched Fable scratch’s 12/12 checks at 89% lower API cost. Haiku stayed incomplete in all arms.

One attempt per condition. Cost bars: \$0–\$5. Haiku 4.5 / Sonnet 5.5 / Fable 5.1. Rhiza supplied catalog context; scripts were not called.

Follow-up task: forecast the next two years

Produce Oct–Dec rainfall probabilities for 2007–08 in the same Kenyan study area (initial: 2005–06).
Fable, fresh conversation: reuse the saved model without refitting; validate the workflow and record repairs.



ACCORD’s follow-up: 50% fewer tokens, 50% lower cost and 31% less time than scratch.

Both passed 12/12 checks. Across both attempts, ACCORD cost 12% less.

RPS ↓: scratch 0.396; Rhiza 0.404; ACCORD 0.412. No accuracy advantage established. No fresh-workspace controls or download savings.

How good were the forecasts?

Fable in all three arms, evaluated against CHIRPS observations.

Ranked Probability Score (RPS): error in below / near / above probabilities; lower is better.

Forecast	2005–06 initial	2007–08 follow-up
Raw ECMWF	0.369	0.439
Simple model-climatology baseline	0.296	0.353
Fable / scratch	0.279	0.396
Fable / Rhiza	0.281	0.404
Fable / ACCORD	0.279	0.412

All three beat raw ECMWF; initial forecast accuracy is similar across arms.

The simple baseline wins on the follow-up. No ACCORD accuracy advantage established.

Two previously inspected years per batch; general skill and calibration remain unproven. Rhiza’s predictor scored, but workflow replay failed.

Toward auto-science

Make each experiment leave behind something the next agent can use.

Research → Hypothesis → Experiment → Evaluate

Keep tested code, data, fitted models, findings and failed attempts.

Use that shared record to choose and run the next experiment.

Today: measure completion, cost and reuse. Next: test whether scientific improvements accumulate.